Survey and Empirical Evaluation of Attacks and Defense for Robust Classification

Lirui Wang
University of Washington
Seattle, USA
lirui@cs.washington.edu

Abstract

Challenges to neural network classifiers these days involve corruptions at training time and at testing time. Data corruptions and class imbalance at training time can heavily degrade the generalization and robustness performance. Data corruption occurs at testing time can drastically change neural net prediction with an imperceptible amount of perturbations. As a consequence, data overfitting and adversarial examples present threats to security sensitive applications such as self-driving car. In this work, we evaluate example reweighting and adversarial training techniques to robustify a fossil classification task. Additionally, we survey recent papers about attacks and defense of adversarial examples for neural networks.

1 Introduction

Deep nets have shown to be powerful classifiers in various settings. However, data corruptions present a challenge to robustness and generalization. Corruptions at training time are often considered as learning with the presence of outliers. More generally, it's caused by the distribution shift between training set and test set. Class imbalance and label noise in the training set are two common problems that lead to poor generalization performance. Example reweighting algorithms are popular solutions to these problems, but most methods require careful tuning of additional hyperparameters, such as example mining schedules and regularization hyperparameters. We investigate a meta-learning algorithm [16] that learns to assign weights to training examples based on the performance on a clean validation dataset.

Corruptions can also happen in test time that act as adversarial examples to neural networks. The adversarial attacks and defense have been researched extensively these three years [20; 6; 11]. Attacks can be white-box that takes detailed model structure information with gradient or black-box that only requires model oracle calls. Associated with a perturbation set, white-box attack methods such as projected gradient descent (PGD) performs a first order update to the objective of maximizing classification loss. [11] demonstrates that this is the natural and "universal" attack with gradient information. [1] generates attacks that stably fool neural nets under natural transformations. [2] propose an network trained to generate white-box attacks. Transferable attacks are commonly used in white-box adversaries where a proxy is trained and attacked first and later adapted. An adversarial subspace is also proposed to analyze the transferability of attacks. Correspondingly, [19] and [9] use diverse ensemble to improve the robustness to adversarial attacks. Finally, [7] and [3] use only oracle calls to DNN to generate attacks efficiently.

For the empirical evaluation, we consider a fossil classification task on 224×224 grayscale images with total 6000 images and 33 classes. The job is to maximize classification accuracy on the test set, to classify the tribe of a given fossil image. Class imbalances and label noises are often present on this setting as the fossil data can be hard to collect and label correctly by humans. Moreover, test time robustness can be extremely important as a false fossil classification might mistakenly bias a biology researcher. Our experiments on reweighting examples reproduce the results on MNIST

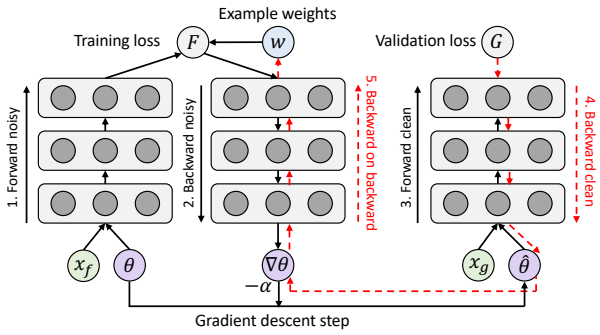

Figure 1: To get the gradients of the example weights, one needs to first unroll the gradient graph of the training batch, and then use backward-on-backward automatic differentiation to take a second order gradient pass.

dataset and show a performance improvement on fossil dataset. We also demonstrate comparable results with hyperparameters tuning. For test time corruption, we first produce attacks with PGD to easily fool the trained network. We then show a large robustness improvement with adversarial training. Additionally, we investigate more general methods to generate adversarial examples to attack deep nets in real world.

2 Meta Reweighting

The key idea of meta reweighting based on a clean validation set is that example weighting should minimize the loss of a set of unbiased clean validation examples that are consistent with the evaluation procedure. More formally, denote (x_i, y_i) be the i th input-target pair in the training set and (x_i^v, y_i^v) be the i th input-target pair in the clean validation set. Denote θ be the neural network parameters and $f_i(\theta)$, $f_i^v(\theta)$ be the loss on the i th training example and validation example respectively.

The goal is to learn a reweighting of the inputs to minimize a weighted loss:

$$\theta^*(w) = \arg\min_{\theta} \sum_{i=1}^{N} w_i f_i(\theta) \tag{1}$$

with w_i unknown upon beginning. The optimal selection of w is based on its validation performance:

$$w^* = \arg\min_{w \geq 0} \frac{1}{M} \sum_{i=1}^{M} w_i f_i^v(\theta^*(w)) \tag{2}$$

Consider perturbing the weighting by ϵ_i for each training example in the batch

$$\begin{aligned} f_{i,\epsilon}(\theta) &= \epsilon_i f_i(\theta) \\ \hat{\theta}_{t+1}(\epsilon) &= \theta_t - \alpha \nabla \sum_i f_{i,\epsilon}(\theta)|_{\theta=\theta_t} \end{aligned} \tag{3}$$

Then we can take a single gradient descent step on a mini-batch of validation samples with respect to ϵ_t to online approximate the minimizer

$$\epsilon_t^* = \operatorname*{argmin}_{\epsilon} \sum_{i=1} f_i^v(\theta_{i+1}(\epsilon)) \tag{4}$$

This method can be easily implemented with standard autodifferentiation package [15], as shown in the Fig. 1. Note that the meta example reweighting schedule is different from finetuning in that the clean validation acts more like a regularizer rather than a data source. Since the weight ϵ_t regularizes the learning rate at each iteration, it's not surprised to see a performance improvement.

We first reproduce its result on a highly imbalanced MNIST subset containing 5000 images with 25 images of classes 4 and 4975 images of 9. The network, LeNet in this case, is able to learn

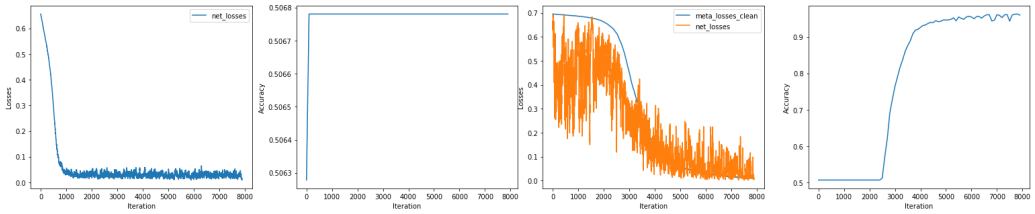

Figure 2: On imbalanced MNIST dataset, (left) it's not surprised that normal training on such imbalanced dataset won't generalize to any meaningful classification. Indeed it reaches 50 percent on test set, the same as random guessing. (Right) The meta example reweighting algorithm helps the network learn to differentiate between 4 and 9 and achieve an accuracy as high as 95 percent. This implies the training takes advantage of the clean validation set as well as the size of the training set.

something nontrivial compared with normal training. In our experiments, we use meta-reweighting to finetune the ImageNet-pretrained network on our fossil dataset. On our harder fossil dataset, shown in Table 1, there's a nontrivial improvement using example-reweighting. However, a hand-tuning hyperparameter (momentum and weight decay) with a more advanced optimizer (ADAM) can reach comparable results. Furthermore, meta-reweighting takes up large GPU resource and time with a relatively large validation set. We were unable to finish the training experiments with VGG11 on a clean validation size 500 and a NVIDIA TITAN X.

3 Adversarial Training

A very popular type of adversarial attack against deep networks is projected gradient descent (PGD). For an example pair (x, y), since neural network classifier has logistics as its final layer activation, a natural attack is to simply find a perturbation which maximizes the logistic loss of x, subject to the constraint that it lives within P_x

$$x^* = \arg\max_{x' \in P_x} L(f(x'), y) \tag{5}$$

let π be the projection onto P_x, for some step-size schedule η_t, we can define the sequence of updates $x_0 = x$, and

$$x_{t+1} = \pi(x_t + \eta_t \nabla_t L(f(x_{t+1}), y)) \tag{6}$$

This gradient ascent update does well in attacking normally trained neural network. Moreover, PGD can be targeted to specific class or untargeted to just modify the network prediction. While the input image is perturbed in imperceptible amount from human eyes (Fig. 3), the neural nets can be easily fooled.

A common empirical defense is called adversarial training, trying to run SGD on the robust loss of the classifier directly [11]. The gradient of the robust loss of f can be written as

$$\mathbb{E}_{(x,y)\sim\mathcal{D}}[\nabla_\theta(\sup_{\delta \in \mathcal{P}_{p,\epsilon}(0)} L(f(X+\delta), y))] \tag{7}$$

By Danskin's theorem, to evaluate the gradient of the supremum of a class of functions, one should simply evaluate the gradient of the function in the class that actually obtains the maximum. So, an iteration of the adversarial training can be run with adversarial attack, such as PGD. In a high level, the adversarial training plays a role in robustifying decision boundary in the embedding space. As shown in Table 1, we evaluate the performance of various training with architecture ResNet18 [18] and VGG11 [8] on adversarial attacks [5]. The adversarial examples are generated with different constraint size by an attack learning rate of 0.1 for 10 steps on untargeted class.

4 More Empirical Attacks and Defence

While PGD can generate adversarial attacks to a single network efficiently, it requires detailed model structure with first order information and is not invariant to viewpoint shifts, camera noise, and other natural transformations. This is also true for other optimization-based approaches such as Fast

Evaluation of robustness to adversarial attacks using PGD						
Network	Resnet				VGG	
Stat./Alg.	Baseline	Tune.	Reweight	Adv.	Tune.	Adv.
L_2: Clean	80.76%	84.91%	**86.97**%	81.06%	82.88%	78.48%
L_2: 0.001	77.42%	85.00%	**85.61**%	81.06%	82.58%	78.03%
L_2: 0.01	51.36%	74.70%	76.06%	**79.39**%	78.18%	76.52%
L_2: 0.1	0%	4.85%	14.24%	**51.52**%	17.12%	40.61%
L_{inf}: Clean	80.76%	85.91%	**86.97**%	84.09%	82.88%	66.67%
L_{inf}: 0.00001	75.76%	84.39%	**85.61**%	83.79%	82.42%	63.79%
L_{inf}: 0.0001	36.82%	69.39%	73.48%	**80.91**%	77.27%	61.52%
L_{inf}: 0.001	0%	5.45%	13.48%	**55.91**%	18.03%	46.52%

Table 1: We evaluate the robustness of each trained network on adversarial examples using PGD on L2 and L-infinity norm constraints. Baseline denotes normal training schedule to minimize loss on training set. Tune denotes tuning on hyperparameter for training. Reweight denotes meta example reweighting method. Adv. denotes adsersarial training. From the result, we see a huge improvement by adversarial training on adversarial attacks and an improvement of generalization of reweighting procedure. We also note that reweighting and larger capacity network [11] such as VGG provides some defense to adversarial attacks.

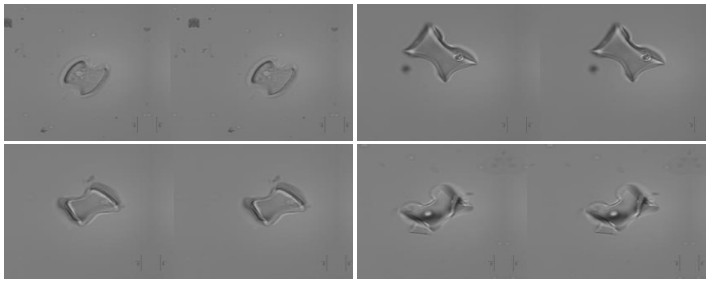

Figure 3: (Left) Original image. (Right) Example adversarial image generated PGD with perturbation constraint 0.001 in infinity norm, i.e the grayscale value of these two images do not differ by 0.001 per pixel (0.2 pixel in 256 pixel scale). For instance, on the bottom left, the left image is correctly classified as guaduellieae, and the right one is predicted as arundinarieae, while they look exactly the same to human eyes.

Gradient Sign Method or Iterative Attacks [6]. In this section, we explore a few empirical methods for robust white-box attacks and black-box attacks with transfer training and zeroth order optimization.

4.1 Expectation Over Transformation for Robust PGD Attack

White-box adversarial examples compute gradients from the loss function with respect to the input pixel. However, The perturbations carefully crafted by standard techniques often fail To synthesize robust adversarial examples that are relevant in real world [10], due to natural phenomena such as lighting and camera noises. [1] propose Expectation Over Transformation (EOT) algorithm to construct attacks that remain adversarial after transformation t from a chosen distribution T, as an example shown on Figure. Moreover, for imperceptible perturbation, EOT aims to constrain the expected effective distance between the adversarial and original inputs,

$$\delta = \mathbb{E}_{t\sim T}[d(t(x'), t(x)] \tag{8}$$

For instance, for a rendered 3D image, this would minimize the visual difference instead of the difference in texture space. In practice, they use distance in LAB space as a proxy for human perceptual distance. The distribution of transformation apply differently to 2D and 3D cases. In 2D, T includes rescaling, rotation, lightening, and additional noises. In 3D, camera distances, lighting conditions, translation and rotation, and texture difference are considered. The overall soft-constrained objective is defined as

$$\underset{x'}{\operatorname{argmax}}\, 0_{t\sim T}[\log P(y_t|t(x')) - \lambda \left\| LAB(t(x')) - LAB(t(X)) \right\|] \tag{9}$$

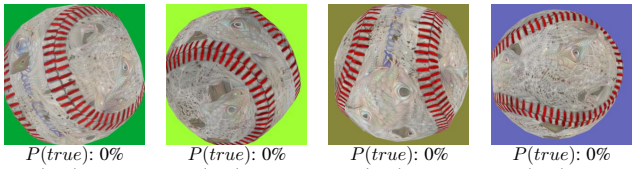

Figure 4: [1] The synthesized robust adversarial example to fool a classifier predict a baseball (P_{true}) as a green lizard (P_{adv}).

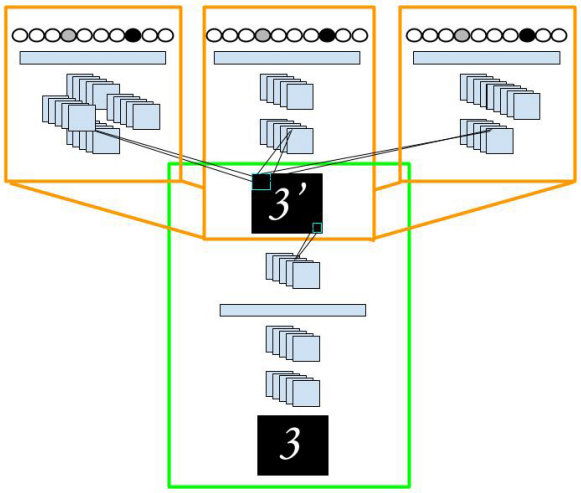

Figure 5: [2] ATN takes input (clean image 3) and transform it into an adversarial example to a set of target networks, and fool them to predict 7.

4.2 Adversarial Transformation Networks

[2] propose an Adversarial Transformation Networks (ATN) to learn to generate adversarial examples. Similar to GAN, ATN is a neural network that transforms an input into an adversarial example against a target network or set of networks. Define ATN as

$$g_{f,\theta}(x) : x \in \mathcal{X} \rightarrow x' \tag{10}$$

where θ is the parameter vector of g, f is the target network which outputs a probability distribution across class labels, and $\operatorname{argmax} f(x) \neq \operatorname{argmax} f(x')$. The training objective is defined as

$$\underset{\theta}{\operatorname{argmin}} \sum_{x_i \in \mathcal{X}} \beta L_{\mathcal{X}}(g_{f,\theta}(x_i), x_i) + L_{\mathcal{Y}}(f(g_{f,\theta}(x_i)), f(x_i)) \tag{11}$$

where $L_{\mathcal{X}}$ is a loss function in the input space, $L_{\mathcal{Y}}$ is a specially-formed loss on the output space of f (described below) to avoid learning the identity function, and β is a weight to balance the two loss function. This is trained in a white-box fashion and requires no access to the network at test time. The transformed input can be targeted to particular wrong class t with a designed reranking function $r(y, t)$ for $L_{\mathcal{Y}}$ [2]. Adversarial example can be generated by Perturbation ATN (P-ATN) or Adversarial Autoencoding (AAE). In P-ATN, ATN acts as a variation on the residual block, while AAE attempts to reconstruct the original input subject to the perturbation.

4.3 Transferability for Black Box Attacks

While ATN works as black-box attack at test time, it's only trained for a particular set of networks. Adversarial examples that affect one model often affect another model, even if different architectures

Transferability of adversarial attacks.						
Network	Resnet				VGG	
Stat./Alg.	Baseline	Tune.	Reweight	Adv.	Tune.	Adv.
L_2: Clean	80.76%	84.91%	**86.97**%	81.06%	82.88%	78.48%
L_2: 0.1	76.82%	**81.06**%	71.36%	51.52%	74.09%	78.48%
L_{inf}: Clean	80.76%	84.91%	**86.97**%	84.09%	82.88%	66.67%
L_{inf}: 0.001	73.79%	**81.36**%	70.61%	55.91%	74.39%	66.82%

Table 2: We generate adversarial examples with L2 and L-infinity norm constraints to adversarial trained ResNet. We then use those examples to test the transferability of adversarial attacks to our fossil classification tasks. It turns out that the examples generated transfer not very well as most accuracy for other model weights and architecture do not decrease by much.

or different training sets are used, so long as both models perform the same task. Therefore, an attacker may train a substitute model, craft adversarial examples against the substitute, and transfer them to a victim model [13]. Since black box attacks only assume oracle calls to neural network outputs, adversary can even synthetically generate a dataset and labeled by the target DNN. After that, they can use local substitute to craft adversarial examples for the target DNN [14]. [13] showed that machine learning approaches are in general vulnerable to systematic black-box attacks regardless of their structures, such as SVM, kNN, or DNN. They show effective attacks on standard model publicly available on Google and Amazon with 700 queries. In our experiments on transferability of the adversarial examples generated by PGD (shown in Table 2), adversaries for adversarially trained ResNet do not act as a large threat to other trained networks in our experiments.

4.4 Adversarial Subspace for Defense and Attacks

It has been shown that models with high dimensionality of adversarial subspace [20] are more likely to intersect, causing adversarial examples to transfer between models. It's natural to explore the attacks and defense of ensemble adversarial training because the attack vector must now fool multiple models in the ensemble instead of just a single model, Moreover, [19; 9] use experiments to show that ensemble of networks increases the diversity of perturbations seen during training and decouples adversarial example generation from the parameters of the trained model. Ensemble Adversarial Training augments a model's training data with adversarial examples crafted on other static pre-trained models, which relies on the transferability among models. [9] proposes gradient alignment loss (GAL) to train an ensemble of diverse models with misaligned gradients, thus reducing the dimensionality of the shared adversarial subspace. Intuitively, if the gradients of two models are misaligned, the perturbations that cause J_0 to increase do not cause J_1 to increase. [9] define the cosine similarity between gradients

$$CS(\nabla_x J_0, \nabla_x J_1) = \frac{\langle \nabla_x J_0, \nabla_x J_1 \rangle}{|\nabla_x J_0||\nabla_x J_1|} \tag{12}$$

The GAL loss is defined as the smooth approximation to max operator

$$GAL = \log\big(\sum_{1 \le a < b \le N} \exp(CS(\nabla_x J_a, \nabla_x J_b \rangle))\big) \tag{13}$$

They propose using the GAL cost as a regularization term for Diversity Training.

4.5 Zeroth Order Attacks

To generate black-box adversarial attacks without transfer learning, several papers have proposed zeroth order optimization with only oracle calls. These methods [3; 12] often use numerical approximations to compute higher order information such as gradients and hessians, and achieve effective attacks on datasets such as CIFAR and MNIST. [3] directly estimate the gradients of the targeted DNN for generating adversarial examples. Specifically, their ZOO algorithm uses stochastic coordinate descent and effectively attack DNN and speed up computation time and reduce number of queries by attack-space dimension reduction, hierarchical attacks and importance sampling. They use numerical approximation to compute gradient and hessian of a random coordinate, and apply different algorithm such as ADAM and Newton's Method to minimize the softmax probability of the predicted class.

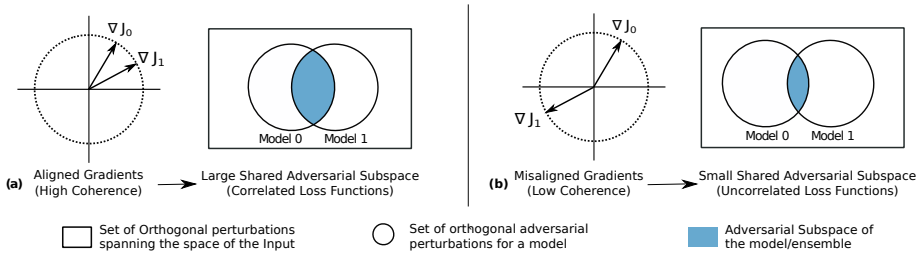

Figure 6: [9] Illustration showing the relationship between the gradient alignment and overlap of adversarial subspaces of two model. Diverse Training aims to train an ensemble with a smaller overlap in the adversarial subspace.

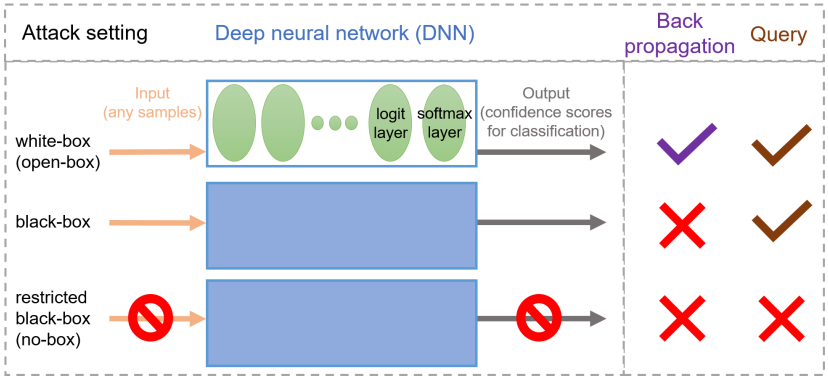

Figure 7: [3] Taxonomy of adversarial attacks to deep neural networks. Zeroth order optimization (middle) requires query of the network but not the gradient information.

They use dimension reduction techniques to reduce the number of estimated gradients and use a hierarchical attack scheme to gradually increase the number of applied transformation. They also propose to optimize the important pixels first, for instance, the pixels near the main object, by using importance sampling.

[7] propose a simple black-box attacks guided by output probabilities to search for adversarial images without transferability. Their proposed method Simple Black-box Attack (SimBA), takes as input the target image label pair (x, y), a set of orthonormal candidate vectors Q and a step-size $\eta > 0$. Their algorithm is then simple as the intuition: for any direction $q \in Q$ and some step size η, one of $x + \eta q$ or $x - \eta q$ is likely to decrease $p(y|x)$. Their algorithm can be therefore implemented easily and demonstrated effective attacks to models without any extra information than oracle calls.

5 Conclusion

In this work, we evaluate two training methods, meta example-reweighting and adversarial training, on fossil dataset to address corruptions at training time and test time. Moreover, we survey current empirical methods on generating black-box and white-box attacks to DNN, with a special focus on PGD robust to transformation, adversarial transformation network, transferability, adversarial subspace, and zeroth order attacks. These methods have shown effectiveness in defending and attacking neural networks for certain tasks in certain settings. As certified defense methods are developed to be more applicable in large scale [17; 4] and show provable bounds, it would be interesting to compare and analyze those methods with empirical methods.

References

[1] Anish Athalye, Logan Engstrom, Andrew Ilyas, and Kevin Kwok. Synthesizing robust adversarial examples. *arXiv preprint arXiv:1707.07397*, 2017.

[2] Shumeet Baluja and Ian Fischer. Adversarial transformation networks: Learning to generate adversarial examples. *arXiv preprint arXiv:1703.09387*, 2017.

[3] Pin-Yu Chen, Huan Zhang, Yash Sharma, Jinfeng Yi, and Cho-Jui Hsieh. Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, pages 15–26. ACM, 2017.

[4] Jeremy M Cohen, Elan Rosenfeld, and J Zico Kolter. Certified adversarial robustness via randomized smoothing. *arXiv preprint arXiv:1902.02918*, 2019.

[5] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, and Dimitris Tsipras. Robustness (python library), 2019. URL `https://github.com/MadryLab/robustness`.

[6] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

[7] Chuan Guo, Jacob R Gardner, Yurong You, Andrew Gordon Wilson, and Kilian Q Weinberger. Simple black-box adversarial attacks. *arXiv preprint arXiv:1905.07121*, 2019.

[8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[9] Sanjay Kariyappa and Moinuddin K Qureshi. Improving adversarial robustness of ensembles with diversity training. *arXiv preprint arXiv:1901.09981*, 2019.

[10] Jiajun Lu, Hussein Sibai, Evan Fabry, and David Forsyth. No need to worry about adversarial examples in object detection in autonomous vehicles. *arXiv preprint arXiv:1707.03501*, 2017.

[11] Aleksander Mądry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *stat*, 1050:9, 2017.

[12] Arjun Nitin Bhagoji, Warren He, Bo Li, and Dawn Song. Exploring the space of black-box attacks on deep neural networks. *arXiv preprint arXiv:1712.09491*, 2017.

[13] Nicolas Papernot, Patrick McDaniel, and Ian Goodfellow. Transferability in machine learning: from phenomena to black-box attacks using adversarial samples. *arXiv preprint arXiv:1605.07277*, 2016.

[14] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, pages 506–519. ACM, 2017.

[15] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.

[16] Mengye Ren, Wenyuan Zeng, Bin Yang, and Raquel Urtasun. Learning to reweight examples for robust deep learning. In *International Conference on Machine Learning*, pages 4331–4340, 2018.

[17] Hadi Salman, Jerry Li, Ilya Razenshteyn, Pengchuan Zhang, Huan Zhang, Sebastien Bubeck, and Greg Yang. Provably robust deep learning via adversarially trained smoothed classifiers. In *Advances in Neural Information Processing Systems*, pages 11289–11300, 2019.

[18] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[19] Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. Ensemble adversarial training: Attacks and defenses. *arXiv preprint arXiv:1705.07204*, 2017.

[20] Florian Tramèr, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. The space of transferable adversarial examples. *arXiv preprint arXiv:1704.03453*, 2017.